INSTITUTE OF INFORMATION TECHNOLOGY

University of Dhaka

EverBloom

Custom Bouquet & Flower Shop Manager

TECHNICAL DOCUMENTATION

Design Patterns Final Project - Theory & Lab

Team Member	Fahmida Hossain - Roll 1603
Team Member	Arithro Choudhury - Roll 1647
Repository	https://github.com/Avraaaa/Everbloom
Prepared	September 2026

A desktop JavaFX application demonstrating meaningful use of design patterns, persistent SQLite data, multi-step workflows, reporting, testing, and collaborative Git development.

Executive Summary

EverBloom is a desktop flower-shop management application built with Java 21, JavaFX 21, Maven, SQLite/JDBC, and JUnit. It supports catalogue and customer management, custom bouquet construction, reusable templates, optional extras, pricing policies, order placement, lifecycle processing, notifications, event packages, search, and analytical reports. The system is intentionally more than CRUD: its main workflows require object construction, interchangeable pricing, lifecycle-dependent behavior, independent reactions to state changes, undo/redo, and recursive package composition.

The final design uses eight GoF patterns. Six are core patterns - Builder, Prototype, Decorator, Strategy, State, and Observer - and two optional patterns, Command and Composite, were retained only after the finished editor and event-package workflow created genuine needs for them. The architecture separates JavaFX controllers, application services, domain/pattern logic, repositories, and SQLite persistence so that the important business behavior can be tested without launching the UI.

> **Submission alignment:** The repository contains the complete Maven project, SQLite schema/seed logic, source code, tests, UML class diagrams, ER diagram, screenshots, and supporting documentation. This consolidated PDF is the technical-documentation artifact intended to accompany the GitHub repository URL.

Table of Contents

1. Project Overview

1.1 Background and Problem Statement

Flower-shop orders combine many choices that change from customer to customer: flowers and quantities, wrapping, message text, extras, discount policy, fulfilment method, delivery address, and preparation state. Large constructors, repeated conditionals, and ad-hoc UI logic make those combinations difficult to maintain. EverBloom treats these as design problems rather than merely database forms.

Core domain entities are Customer, Flower, Extra, BouquetTemplate, Order, Arrangement, and Notification. These entities support stable order history, reusable designs, controlled order processing, safe editing recovery, and nested wedding/event packages. These requirements drive explicit domain behavior, persistent snapshots, and patterns that manage construction, variation, state, reactions, reversibility, and recursive composition.

1.2 Target Users

- Flower-shop staff who create and price bouquet orders.
- Staff who maintain the catalogue, reusable templates, extras, and customer records.
- Order-processing staff who move orders through preparation and fulfilment states.
- Managers who need dashboard summaries and date-range sales/popularity reports.

1.3 Objectives

- Provide a complete desktop workflow from bouquet design to order delivery.
- Use SQLite as an integral persistent store with meaningful constraints and historical snapshots.
- Apply design patterns only where they solve real change or complexity problems.
- Keep presentation, business logic, patterns, and persistence responsibilities separate and testable.
- Demonstrate collaborative development through feature branches, commits, pull requests, and reviews.

1.4 Final Feature Scope

Feature	Implemented capability
Catalogue Management	CRUD and search for flowers and extras, plus reusable bouquet-template management.
Customer Management	CRUD and search, duplicate-phone handling, validation, and details view.
Bouquet Builder	Customer/occasion selection, flowers and quantities, wrapping/message options, live summary, and build validation.
Reusable Templates	Copy a saved template into an independent editable bouquet.
Extras and Pricing	Stack optional extras and apply Standard or Loyalty pricing.
Undo/Redo	Reverse or reapply flower and extra editing actions.
Order Management	Persist orders, reconstruct historical details, search/filter, and advance through lifecycle states.
Notifications and Dashboard	React to successful order-status changes and surface real dashboard metrics.
Reports	Sales/revenue by date plus popular flowers and extras.
Event Packages	Build nested groups and arrangements with recursive totals and persistence.

2. Course Guideline Alignment

The project was selected and developed against the Design Patterns Final Project Guidelines. The following matrix shows how the final repository satisfies the required scope without adding artificial entities or patterns solely to increase counts.

Guideline expectation	EverBloom evidence
Team of two	Fahmida Hossain (1603) and Arithro Choudhury (1647).
Desktop JavaFX application	Six major navigation areas: Dashboard, Bouquet Builder, Orders, Catalogue, Customers, and Reports.
Maven	pom.xml manages JavaFX, SQLite JDBC, JUnit, compiler, Surefire, and JavaFX run plugin.
SQLite persistence	DatabaseInitializer creates/seeds the schema; data persists in data/everbloom.db.
4+ meaningful entities	Ten persistent tables: customers, flowers, extras, bouquet_templates, bouquet_template_flowers, orders, arrangements, arrangement_flowers, arrangement_extras, notifications.
CRUD for at least three entities	Customers, flowers, extras, and bouquet templates have create/read/update/delete workflows.
2+ multi-step workflows	Custom bouquet order, State-based order processing, and nested event-package creation.
2+ search / analytical operations	Order/customer/catalogue search and filters; sales-by-date, popular-flower, and popular-extra reports.
Meaningful design patterns	Eight patterns solve concrete construction, copying, pricing, state, reaction, undo/redo, and tree-composition problems.
Database quality	Primary/foreign keys, constraints, transactions, historical snapshots, and seeded reference data.
Testing and validation	JUnit coverage for pattern behavior, repositories and reporting plus a manual UI/persistence checklist.
Git collaboration	Feature branches, merge commits/pull requests, identifiable contributions from both members.
Required artifacts	Working source, Maven config, SQLite resources, UML class diagrams, ER diagram, screenshots, and documentation.

3. Technology and System Architecture

3.1 Technology Stack

Layer / concern	Technology
Language	Java 21
UI	JavaFX 21.0.2 + FXML + CSS
Build / Dependency Management	Maven
Database	SQLite via sqlite-jdbc 3.53.4.0
Testing	JUnit Jupiter 5.12.2 + Maven Surefire
Version Control	Git and GitHub

3.2 Layered Architecture

EverBloom uses a one-way responsibility flow: JavaFX controllers coordinate user interaction; services enforce use cases and business validation; domain and pattern classes hold behavior; repositories contain JDBC/SQL; DatabaseConnection and DatabaseInitializer manage SQLite access and schema creation. This keeps SQL out of controllers and JavaFX types out of the business/persistence layers.

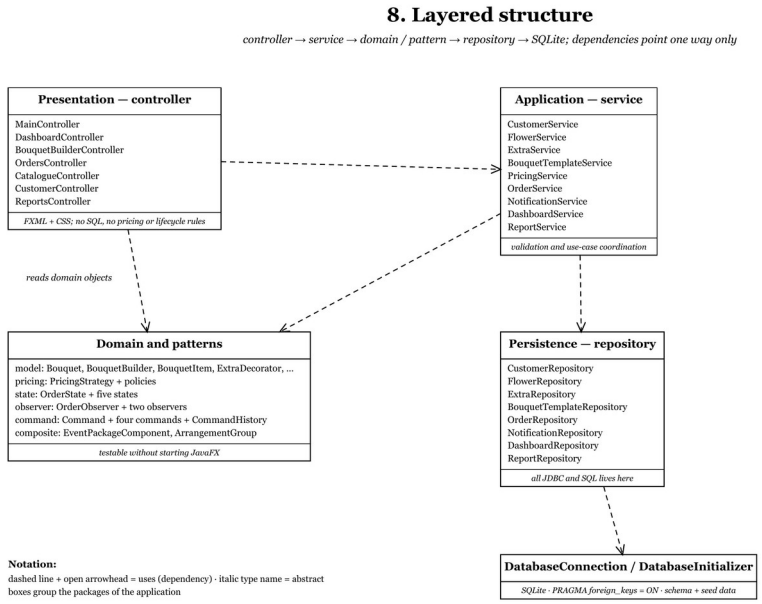


Figure 1. Layered structure of the final application (source: project UML documentation).

3.3 Key Architectural Decisions

Decision	Reason / effect
Layered responsibilities	Controller -> Service -> Domain/Pattern -> Repository -> SQLite. Important behavior can be tested headlessly.
Plain JDBC repositories	PreparedStatement-based repositories keep SQL visible and understandable; multi-table order writes use transactions.
Self-initializing schema	DatabaseInitializer uses CREATE TABLE IF NOT EXISTS and seed logic so a clean clone can start without manual database setup.
Money in minor units	All prices/totals use long integer poisha rather than binary floating point; MoneyFormatter handles display.
Historical snapshots	Orders and arrangement lines store *_snapshot values so old orders and reports remain correct after catalogue changes.

3.4 Source Structure

```
src/main/java/com/everbloom/
  controller/    JavaFX coordination
  service/       use cases and validation
  model/         domain model + Builder/Decorator products
```

```
pricing/       Strategy implementations
state/         State lifecycle classes
observer/      status-change observers
command/       undo/redo commands and history
composite/     event-package tree
repository/    JDBC persistence and reporting queries
database/      connection, schema, seed data
util/          money formatting
```

4. Database and Persistence Design

4.1 Schema Overview

Table	Purpose
customers	Customer identity/contact data; phone is unique.
flowers	Catalogue flower name, color, current unit price, stock, active flag.
extras	Optional add-on name, description, current unit price, active flag.
bouquet_templates	Reusable design metadata such as name, occasion, wrapping and message.
bouquet_template_flowers	Many-to-many template flower composition with quantity.
orders	Customer, fulfilment, status, pricing policy and price snapshots, timestamps.
arrangements	One bouquet or a recursive event-package node; self-reference via parent_arrangement_id.
arrangement_flowers	Historical flower name/unit-price/quantity/line-total snapshots.
arrangement_extras	Historical extra name/unit-price/quantity/line-total snapshots.
notifications	Status-change notification message, status snapshot, read flag, timestamp.

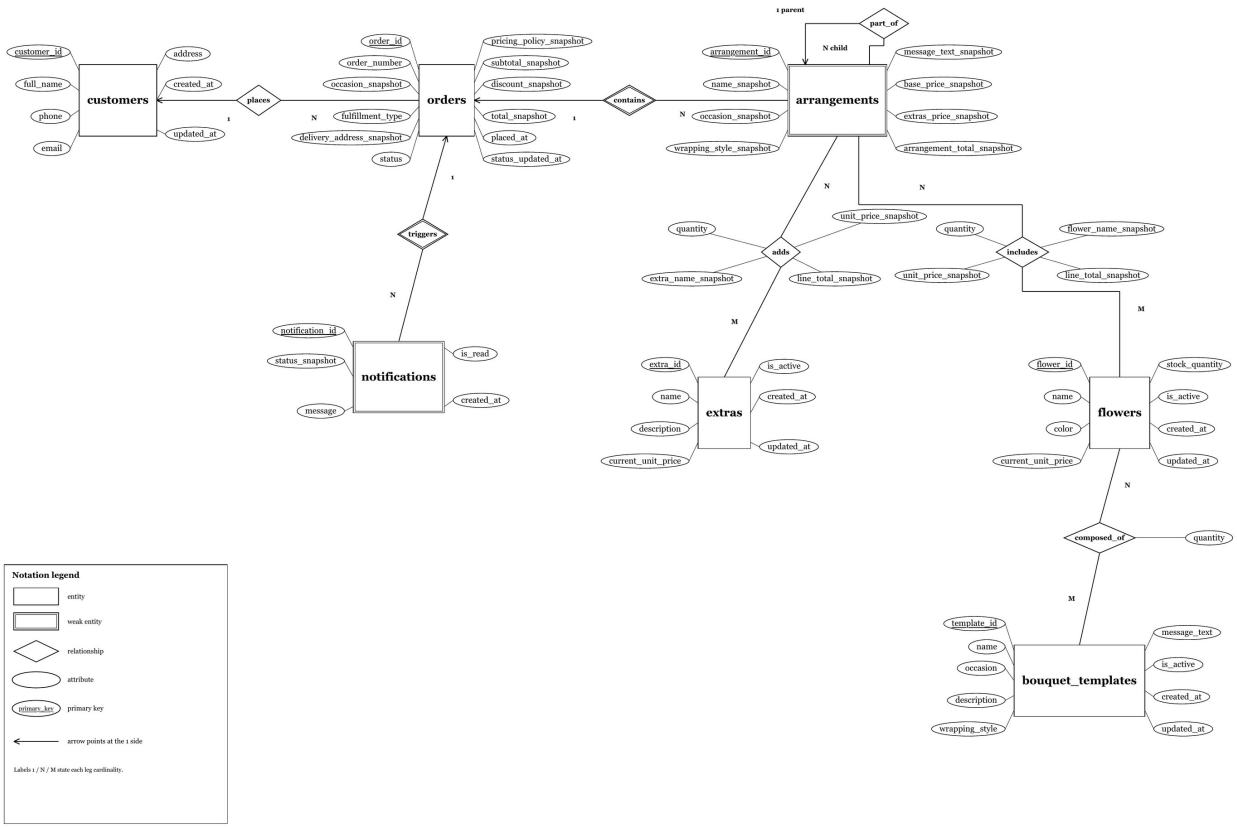
4.2 Integrity, Transactions, and Persistence

- Primary keys, foreign keys, NOT NULL/CHECK constraints, unique values, and deletion behavior protect relational integrity.
- PRAGMA foreign_keys is enabled for SQLite connections.
- Order/event-package persistence uses a transaction so the order, arrangement tree, flower lines, and extra lines commit or roll back together.
- Seed data supplies usable flowers, extras, customers, and reusable bouquet templates without overwriting existing persistent data.
- Snapshot columns preserve the values that were true when an order was placed, which is essential for historical order details and reporting.

4.3 Entity Relationship Diagram

The ER model connects customers to orders, orders to arrangements, arrangements to flower/extra lines, templates to flowers, notifications to orders, and event-package nodes through a self-referencing arrangement relationship.

EverBloom — Entity Relationship Diagram



5. Major Application Workflows

5.1 Custom Bouquet Order

1. Select an existing customer and occasion.
2. Start from scratch or copy a reusable bouquet template.
3. Add flowers and quantities; repeated additions merge into the same bouquet line.
4. Choose wrapping/message options and optionally add extras.
5. Use Undo/Redo for supported flower/extra edits.
6. Select a pricing policy and review subtotal, discount, and final total.
7. Choose pickup or delivery and provide a delivery address when required.
8. Place the order; the system persists historical price/content snapshots and resets the builder after success.

5.2 Order Processing Lifecycle

```
ORDERED -> PREPARING -> ARRANGING -> READY -> DELIVERED
```

Order.advanceStatus() delegates transition behavior to the current State object. OrderService persists a successful transition before notifying observers. Delivered orders reject further transitions, and failed transitions do not create notifications or refresh reactions.

5.3 Event Package Workflow

9. Switch the Bouquet Builder to Event Package mode and create a package root.
10. Add nested groups such as Ceremony, Reception, or Altar.
11. Build bouquet arrangements and attach them to the selected group.
12. Recursive totals and summaries are calculated through Composite.
13. Persist the package tree using arrangements.parent_arrangement_id and reconstruct the same tree in order details.

5.4 Search and Reporting

Orders can be searched by order/customer and filtered by status and fulfilment. Customers support name/phone search; catalogue searches cover flower names/colors and extra descriptions. Reports aggregate persisted snapshot data for sales by date and popularity of flowers/extras within a selected date range.

6. Design Pattern Decisions

Pattern selection was based on actual design problems, not on maximizing the number of patterns. The six core patterns were introduced as the corresponding workflows appeared. Command and Composite remained optional until the finished editor and nested event-package requirement demonstrated clear needs for them.

Pattern	Category	Owner	Real problem solved
Builder	Creational	Fahmida	Step-by-step custom bouquet construction.
Prototype	Creational	Arithro	Copy a reusable template into an independent editable bouquet.
Decorator	Structural	Fahmida	Stack optional extras without subclasses for every combination.
Strategy	Behavioral	Arithro	Swap pricing algorithms without changing checkout.
State	Behavioral	Arithro	Encapsulate lifecycle-dependent valid transitions.

Pattern	Category	Owner	Real problem solved
Observer	Behavioral	Fahmida	Trigger independent reactions after a successful status change.
Command	Behavioral	Fahmida	Undo/redo real flower and extra editing actions.
Composite	Structural	Arithro	Treat nested groups and bouquet arrangements uniformly for recursive totals.

6.1 Builder - Custom Bouquet Construction

Problem

A bouquet is assembled across several UI interactions and contains optional parts. A large constructor cannot be called until editing is complete, while a mutable product could exist in an invalid state.

Implementation and fit

BouquetBuilder accumulates customer, occasion, wrapping, message, and flower selections. It merges repeated flower selections, applies quantity/stock rules, and build() is the final gate that returns a valid Bouquet.

Alternative considered

A six-parameter constructor or mutable setters were considered. Both push partially-built state into the controller or allow invalid products.

Implementation location

`model/BouquetBuilder.java`, `model/Bouquet.java`, `controller/BouquetBuilderController.java`.

Benefit / future change

New bouquet attributes and validation stay in the builder, keeping callers stable and construction logic easy to test.

6.2 Prototype - Reusable Bouquet Templates

Problem

Staff frequently start from a saved design and then customize it. The copied order must be independent so editing one bouquet never mutates the reusable template or another order.

Implementation and fit

BouquetTemplate.copyToBuilder() creates a fresh builder and fresh flower data for the editable copy. BouquetTemplateService exposes the copy-and-customize workflow.

Alternative considered

Passing the template's own mutable flower list directly to the editor was rejected because edits could corrupt the shared design.

Implementation location

`model/BouquetTemplate.java`, `service/BouquetTemplateService.java`.

Benefit / future change

New template attributes can be copied in one place while callers still receive independent editable instances. Future template changes stay localized, reducing shared-state bugs and keeping copy behavior easy to test.

6.3 Decorator - Optional Bouquet Extras

Problem

A bouquet may have any combination of greeting cards, chocolates, ribbons, premium wrapping, and similar extras. Each changes both price and description.

Implementation and fit

BouquetItem defines description/subtotal behavior; BaseBouquet is the base component and ExtraDecorator layers one real extra over the previous component. Decorators can be stacked at runtime.

Alternative considered

A subclass for every combination causes exponential class growth. A raw List<Extra> would also duplicate composition logic between pricing and summary generation.

Implementation location

`model/BouquetItem.java`, `BaseBouquet.java`, `BouquetDecorator.java`, `ExtraDecorator.java`.

Benefit / future change

New extras can be added as decorators without changing bouquet or pricing code, and each extra remains testable.

6.4 Strategy - Pricing Policies

Problem

Checkout needs interchangeable pricing logic: Standard pricing and a Loyalty policy with a discount. New pricing policies should not expand a switch inside checkout.

Implementation and fit

PricingStrategy captures the algorithm. StandardPricingStrategy and LoyaltyPricingStrategy implement different calculations, while PricingService depends on the interface.

Alternative considered

A policy string plus switch is shorter initially but every new policy edits working checkout code and couples tests to the service.

Implementation location

`pricing/PricingStrategy.java`, `StandardPricingStrategy.java`, `LoyaltyPricingStrategy.java`, `service/PricingService.java`.

Benefit / future change

New pricing policies can be added as strategies without changing checkout, and each algorithm remains testable.

6.5 State - Order Lifecycle

Problem

Allowed actions depend on current order status. The lifecycle is ORDERED -> PREPARING -> ARRANGING -> READY -> DELIVERED, and delivered orders must reject further advancement.

Implementation and fit

Order holds an OrderState and delegates advance behavior. Each concrete state knows its allowed successor; DeliveredState enforces the terminal rule.

Alternative considered

A status string and switch in OrderService would mix transition rules with persistence/notification responsibilities and grow with each state.

Implementation location

`state/OrderState.java` and five concrete states; `model/Order.java`; `service/OrderService.java`.

Benefit / future change

New lifecycle states stay in state classes, keeping controllers and repositories stable and transition rules easy to test.

6.6 Observer - Independent Reactions to Status Changes

Problem

After a successful status change, the system must create a notification and refresh the active orders view. These reactions are independent and more could be added later.

Implementation and fit

OrderService maintains a small observer list. NotificationObserver persists a status notification; OrderViewRefreshObserver invokes the UI refresh reaction. Observers run only after a successful persisted transition.

Alternative considered

Direct calls from OrderService to notification persistence and JavaFX callbacks would tightly couple business logic to unrelated components.

Implementation location

`observer/OrderObserver.java`, `NotificationObserver.java`, `OrderViewRefreshObserver.java`; subject behavior in `OrderService`.

Benefit / future change

New SMS, email, or print reactions can be added as observers without changing lifecycle logic or existing reactions.

6.7 Command - Bouquet Undo/Redo

Problem

The finished editor contains real reversible edits: adding/removing flowers and extras. Without undo, a mistaken removal forces staff to reconstruct the bouquet manually.

Implementation and fit

Command exposes execute() and undo(); concrete add/remove commands capture the information required to reverse themselves. CommandHistory maintains undo and redo stacks and clears redo history after a new action.

Alternative considered

Whole-editor snapshots (Memento-style) were rejected because they copy more state than necessary and hide the exact action being reversed.

Implementation location

`command/Command.java`, `CommandHistory.java`, Add/RemoveFlowerCommand, Add/RemoveExtraCommand; controlled by the Bouquet Builder.

Benefit / future change

New reversible edits become commands that reuse the same history, so undo/redo stays stable as the editor grows.

6.8 Composite - Nested Event Packages

Problem

Wedding/event orders need hierarchy: a package contains groups, subgroups, and bouquet arrangements. Totals and summaries must work at every depth.

Implementation and fit

EventPackageComponent is shared by BouquetArrangement leaves and ArrangementGroup composites. Groups recursively sum child totals and build readable summaries. The arrangement tree is persisted with parent_arrangement_id.

Alternative considered

A flat list cannot express group-level totals or nested organization without re-deriving hierarchy outside the model.

Implementation location

`composite/EventPackageComponent.java`, `BouquetArrangement.java`, `ArrangementGroup.java`; event-package persistence in OrderRepository.

Benefit / future change

New package element types can implement the shared component contract and join recursive totals and summaries without changing traversal logic, keeping future hierarchy extensions localized and easier to test.

6.9 Patterns Deliberately Not Forced

Singleton, Factory Method, Abstract Factory, Adapter, Template Method, Visitor, Chain of Responsibility, Memento, and Proxy were considered but not introduced simply to increase pattern count. For example, explicit DatabaseConnection injection is easier to test than a global Singleton, and Command already solves the editor's undo/redo need without a separate Memento subsystem.

7. User Interface and Application Areas

The UI uses a consistent JavaFX/FXML/CSS visual system with a dark-green workspace navigation, light content panels, searchable tables, validation messages, and context-specific actions. The six major navigation areas remain within the recommended 4-8 screen range; order details and event-package controls are integrated into existing areas rather than becoming artificial screens.

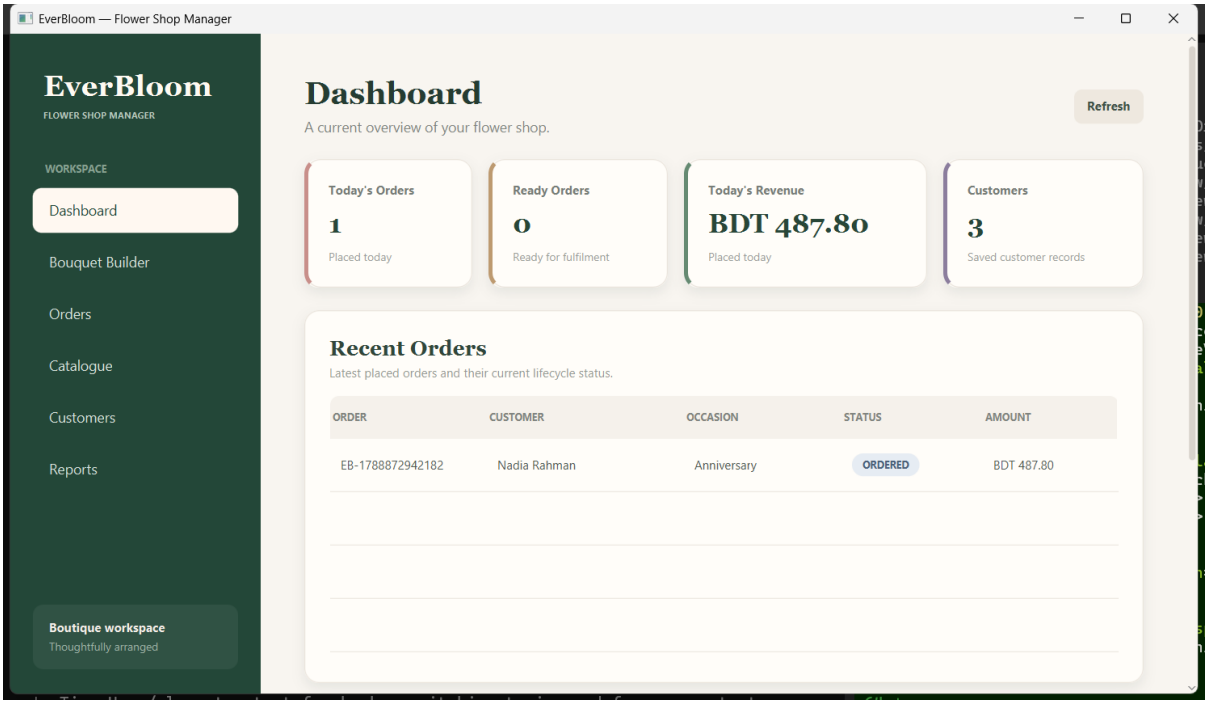


Figure 3. Dashboard with real order metrics, recent orders, and notification feed.

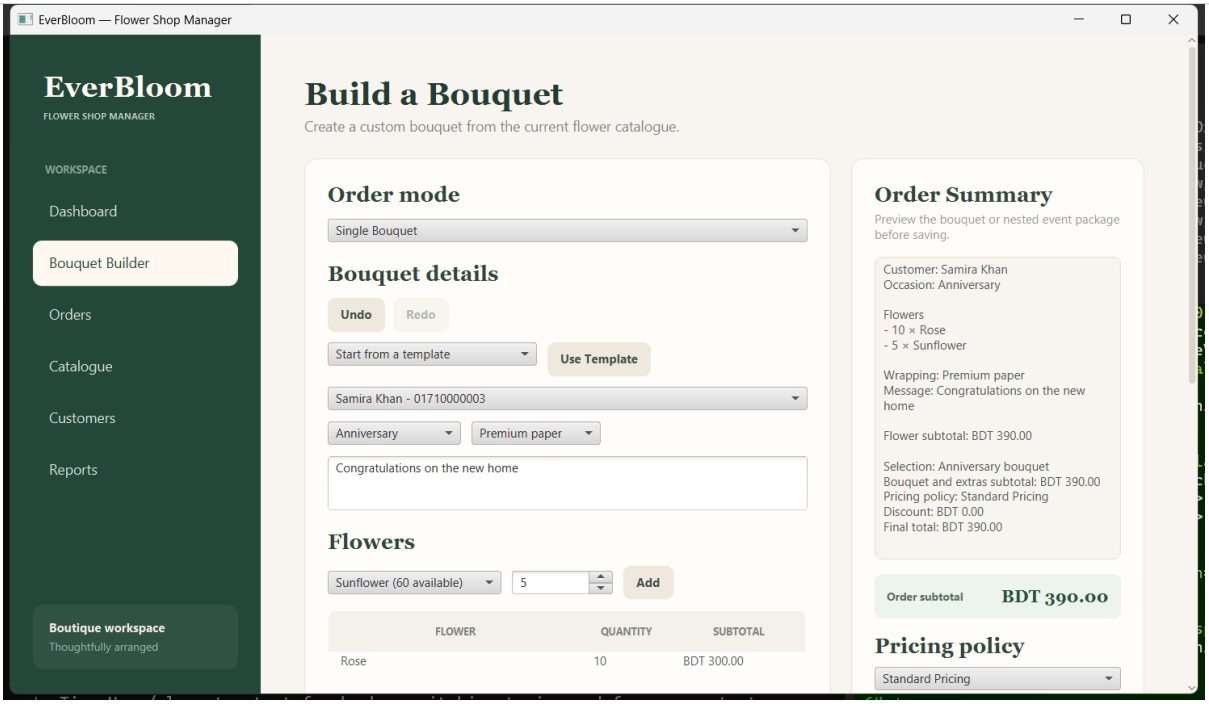


Figure 4. Bouquet Builder: customer/occasion selection, flowers, summary, and editing controls.

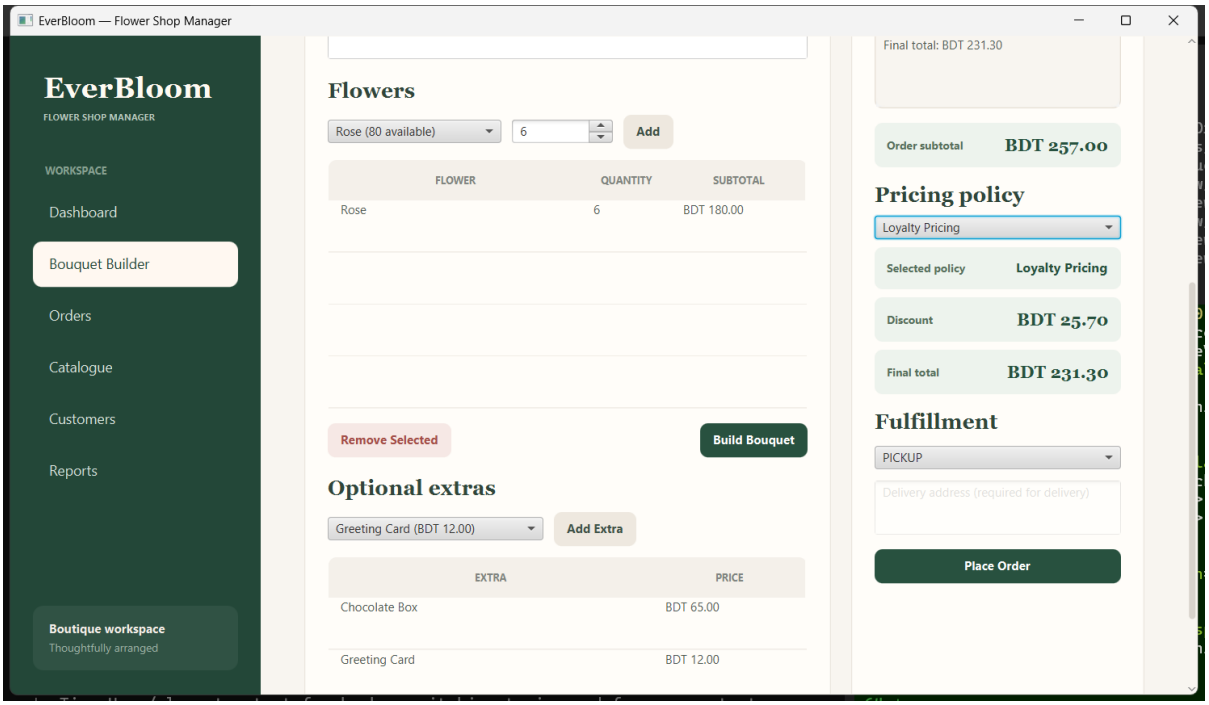


Figure 5. Decorator and Strategy in the UI: stacked extras and Loyalty pricing.

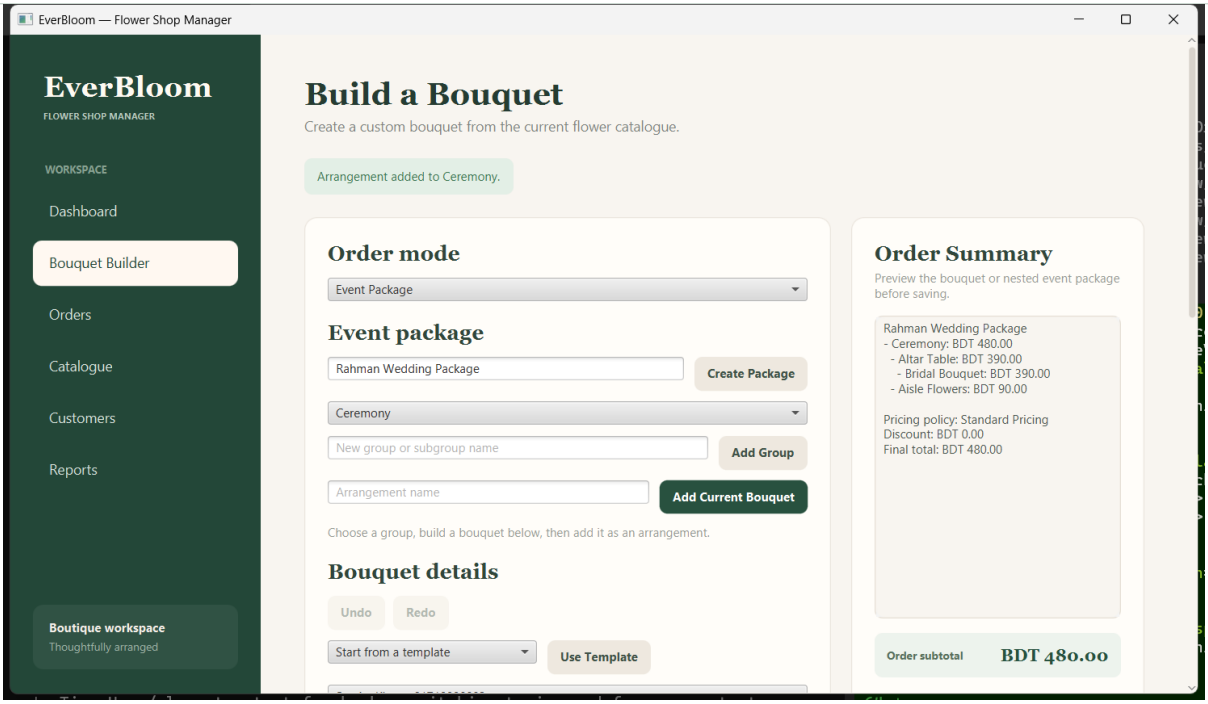


Figure 6. Composite event-package mode with nested group and arrangement workflow.

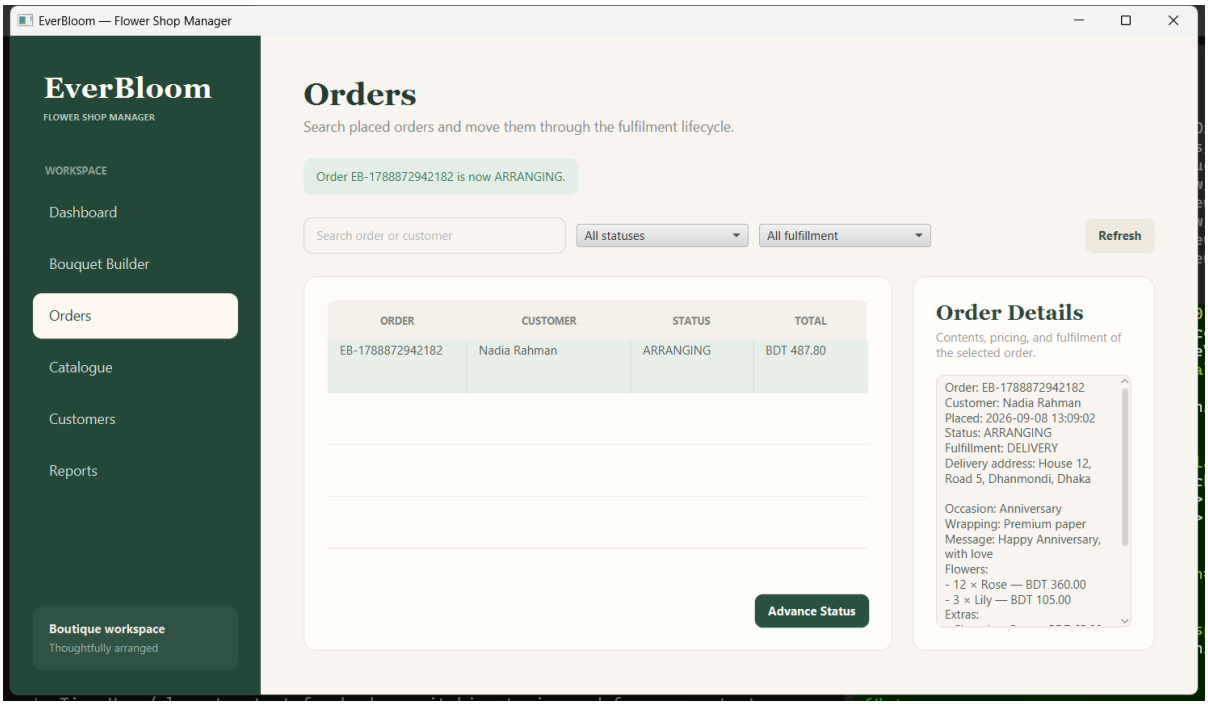


Figure 7. Orders screen during the State-based lifecycle.

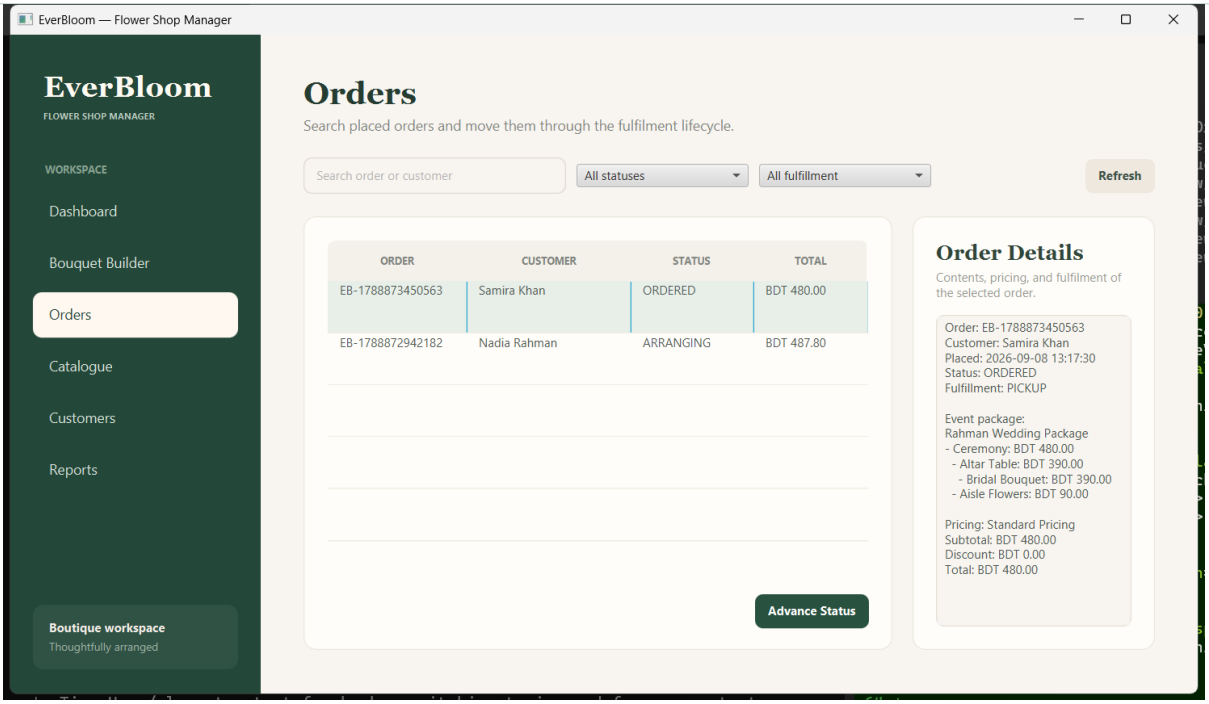


Figure 8. Persisted event package reconstructed in Order Details.

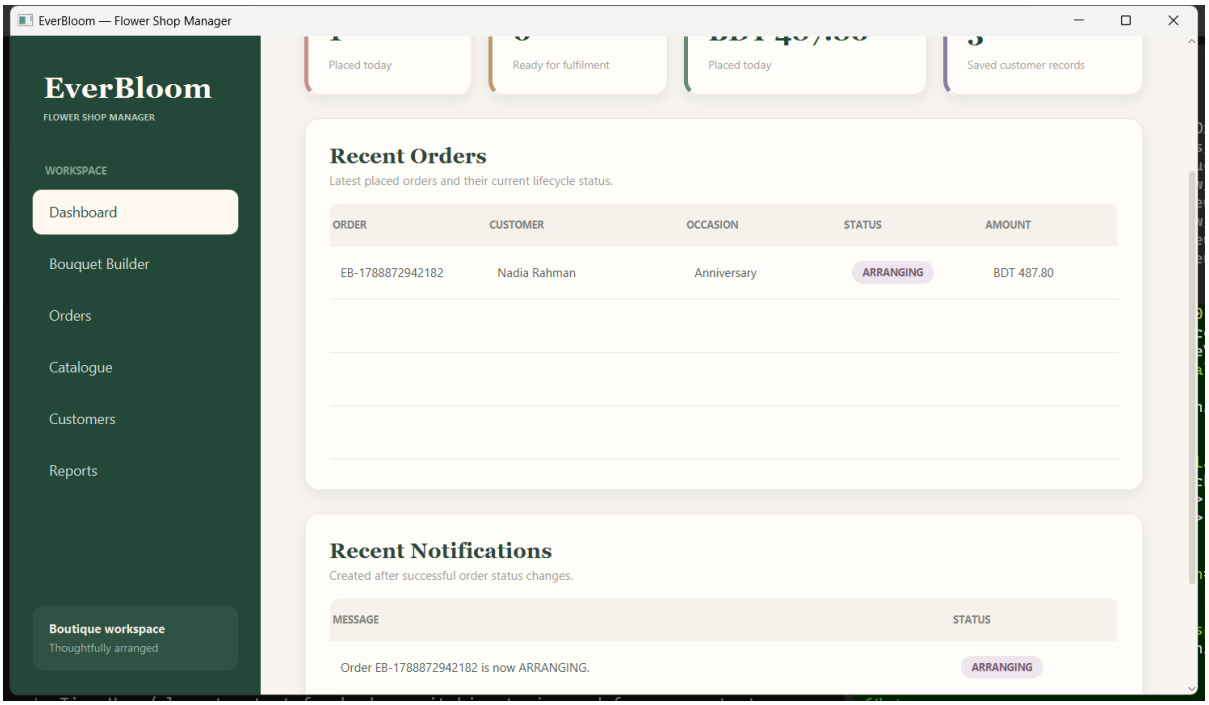


Figure 9. Dashboard notifications produced after successful order-status changes.

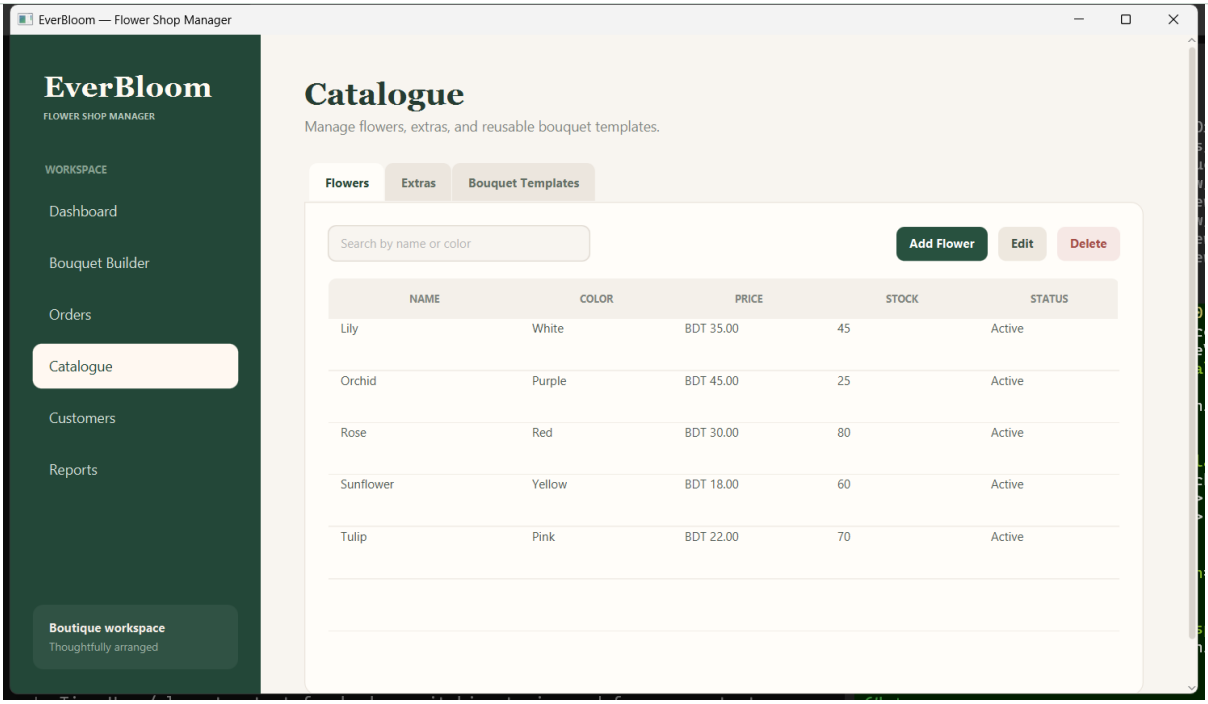


Figure 10. Catalogue management for flowers, extras, and reusable templates.

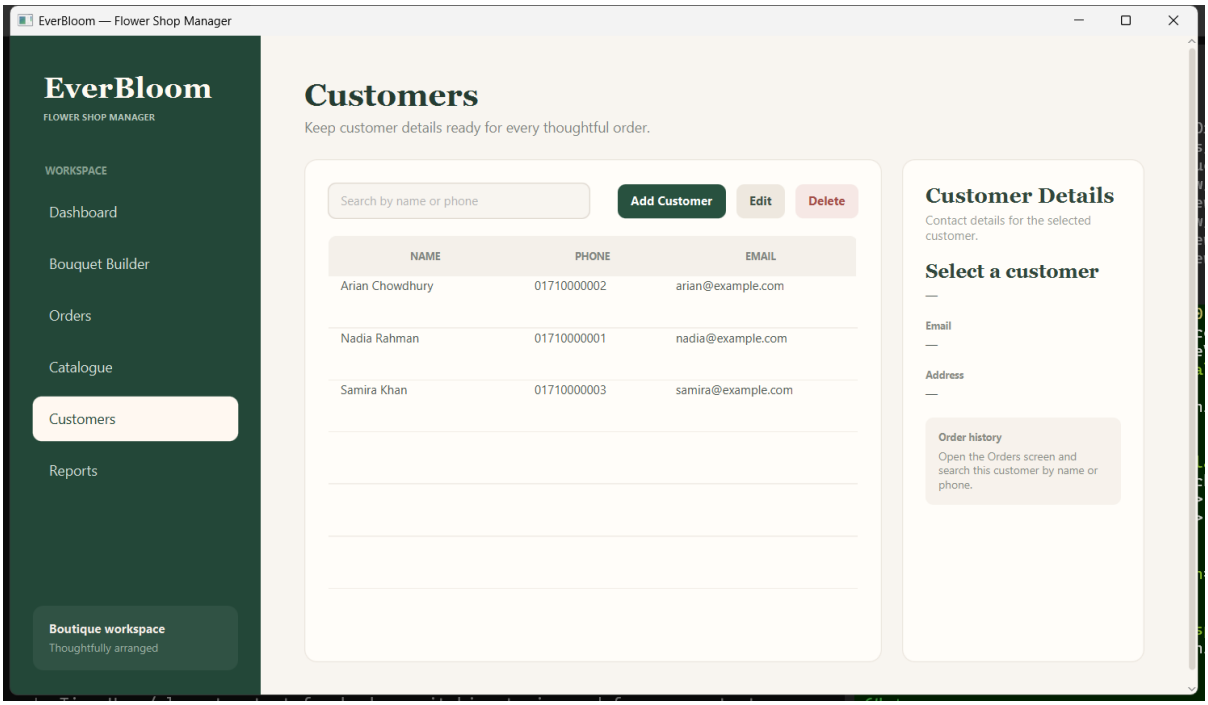


Figure 11. Customer management with search and details panel.

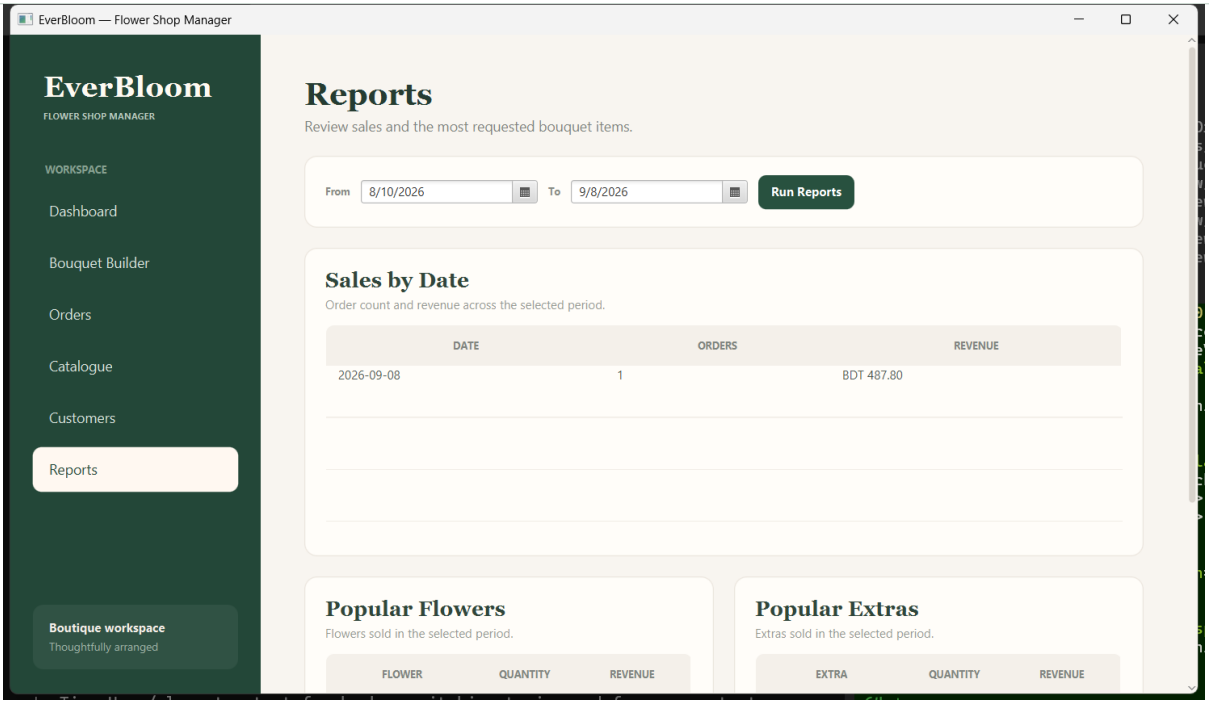


Figure 12. Sales-by-date reporting for a selected range.

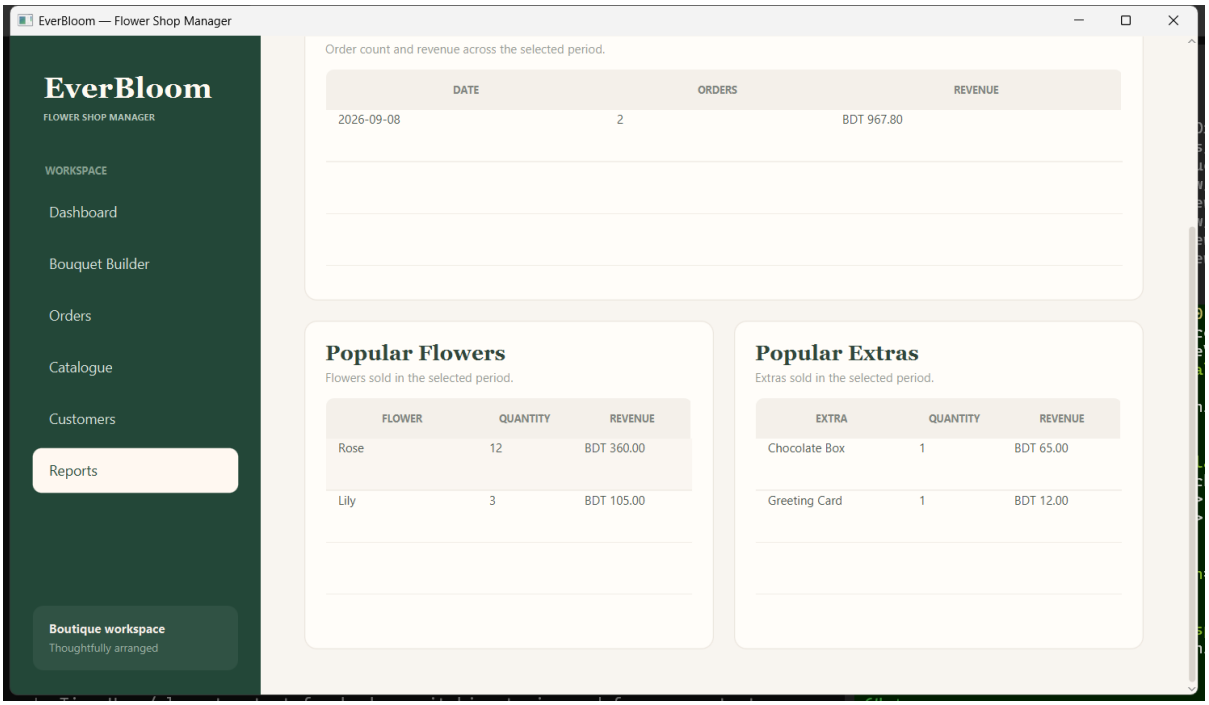


Figure 13. Popular flowers and extras derived from historical order data.

8. Testing, Validation, and Reliability

8.1 Automated Test Coverage

The final repository contains 18 Java test classes with 34 JUnit @Test methods. Tests focus on behavior rather than JavaFX presentation details, allowing core rules and persistence to run headlessly. The Maven workflow uses JUnit Jupiter and Surefire.

Area	Representative test	Behavior protected
Builder	BouquetBuilderTest	Valid construction, invalid construction, quantity/stock rules, independent builds.

Area	Representative test	Behavior protected
Prototype	BouquetTemplateServiceTest	Copy independence: editing one clone does not alter template/other clones.
Decorator	ExtraDecoratorTest	Single and stacked extras change description/subtotal correctly.
Strategy	PricingServiceTest	Standard vs Loyalty calculations and edge behavior.
State	OrderStateTest	Valid lifecycle progression and rejection of invalid terminal transition.
Observer	OrderObserverTest	Successful transitions notify; failed transitions do not.
Command	CommandHistoryTest	Execute/undo/redo, multiple operations, redo-stack behavior.
Composite	ArrangementGroupTest	Nested totals/summary across multiple levels.
Persistence	DatabaseInitializerTest, repository tests	Schema/seed behavior, CRUD, order snapshots, event-package persistence.
Reporting	ReportServiceTest	Date-range sales/revenue and popular item queries.

8.2 Manual Verification

The project documentation includes a manual checklist for JavaFX-specific behavior, persistence after restart, search/filter operations, reports, event-package creation, validation, and presentation consistency. The normal final verification sequence is:

```
mvn clean test
mvn javafx:run
```

8.3 Validation and Error Handling

Area	Representative validation
Bouquet Builder	Requires customer, occasion, at least one flower; prevents duplicate extras; validates delivery address.
Customers	Required name, valid email, unique phone, selection required for edit/delete; protects customers referenced by orders.
Catalogue	Valid price to two decimals, non-negative integer stock, unique flower names, deletion restrictions for referenced records.
Orders	Selection required for status advancement; Delivered rejects further transitions.
Reports	End date cannot precede start date; empty ranges show explicit empty states.
Database	Foreign keys and CHECK constraints protect relationships and snapshot arithmetic.

8.4 Historical Reliability

The system deliberately stores catalogue names and prices as order-time snapshots. Therefore changing a current flower or extra price does not rewrite old order details or historical report totals. Integer minor-unit money also avoids floating-point rounding drift in persisted totals and CHECK constraints.

9. Git Collaboration and Team Contributions

9.1 Development Workflow

Significant work was developed on cohesive feature branches and integrated through merge commits and pull requests/reviews rather than by putting all implementation directly on main. The repository history contains the application foundation/shell, catalogue, bouquet builder, extras/pricing, order management, insights/reporting, undo/redo, event packages, and final-delivery milestones as identifiable commits.

9.2 Contribution Summary

Area	Fahmida Hossain	Arithro Choudhury
Foundation / application shell	Primary implementation	Review/history
Design / database baseline	UI behavior specification	Domain/database baseline
Database foundation	Review/integration	Primary implementation
Catalogue	Flower/Extra data + services	JavaFX Catalogue UI
Customer management	End-to-end implementation	Review
Bouquet core	Builder + builder UI	Prototype + templates
Extras / pricing	Decorator + extras UI	Strategy + pricing integration
Order management	Review/integration	Persistence + State + Orders UI
Insights / reporting	Observer + dashboard/reports	Review/integration
Undo / redo	Command owner	Review
Event packages	Review	Composite owner
Final delivery	Joint review	Final-delivery commit owner

9.3 Pattern Ownership

> **Balanced ownership:** Fahmida owns Builder, Decorator, Observer, and Command. Arithro owns Prototype, Strategy, State, and Composite. Both members also contributed UI, non-UI code, tests, documentation, review, and integration work.

10. Installation and Running

10.1 Prerequisites

- JDK 21.
- Maven 3.x (or the Maven installation configured by the IDE).
- Git for cloning the repository.

10.2 Repository

```
git clone https://github.com/Avraaaa/Everbloom.git
```

```
cd Everbloom
```

10.3 Test and Run

```
mvn clean test
mvn javafx:run
```

On first start, DatabaseInitializer creates and seeds the SQLite database if required. The default application data file is data/everbloom.db, and later launches reuse the persistent data instead of resetting it.

11. Conclusion

EverBloom demonstrates the course objective of using design patterns to manage real software change rather than treating them as isolated textbook examples. The application combines a working JavaFX UI, persistent SQLite database, meaningful multi-step workflows, reporting, validation, tests, and collaborative Git history. Each selected pattern has a clear problem, implementation location, rejected alternative, and extensibility benefit.

The resulting design remains student-scale and explainable: ordinary CRUD and JDBC stay straightforward, while pattern-heavy areas are explicit enough to identify in code and UML during the final demonstration. The repository and this technical document together provide the source, design rationale, evidence, and run instructions required for final evaluation.

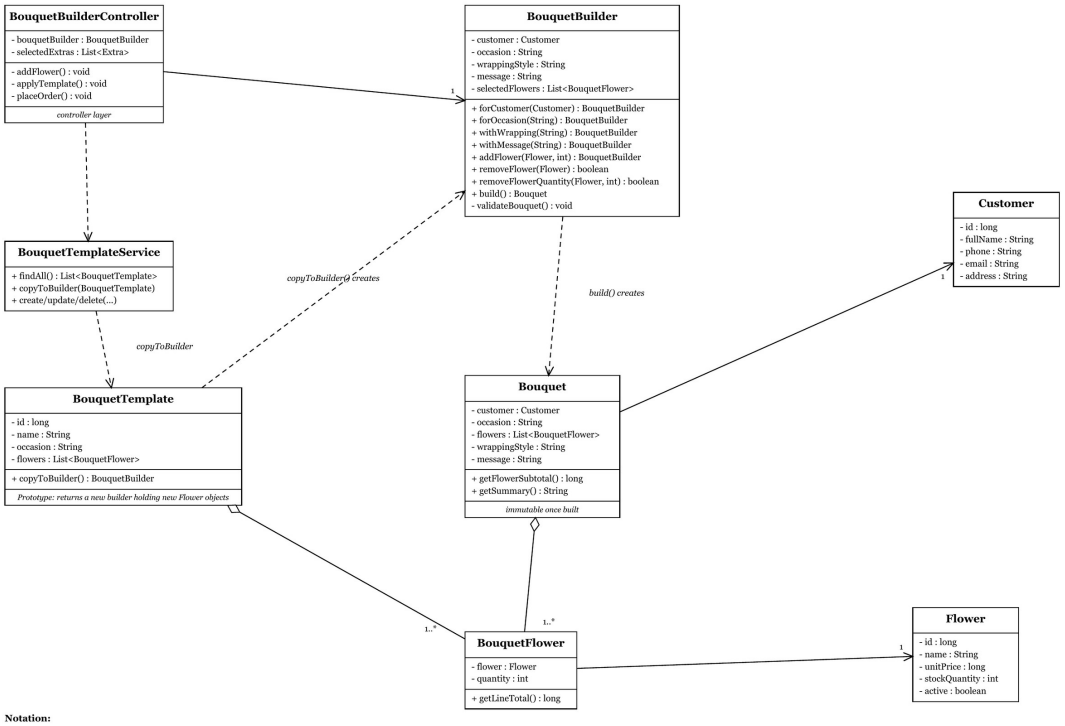
> **GitHub repository:** https://github.com/Avraaaa/Everbloom

Appendix A. UML Class Diagrams

The following diagrams are the project's final UML documentation for the pattern-heavy areas and the layered architecture. They are reproduced from docs/diagrams/uml-class-diagrams.pdf.

1. Builder and Prototype — constructing a bouquet

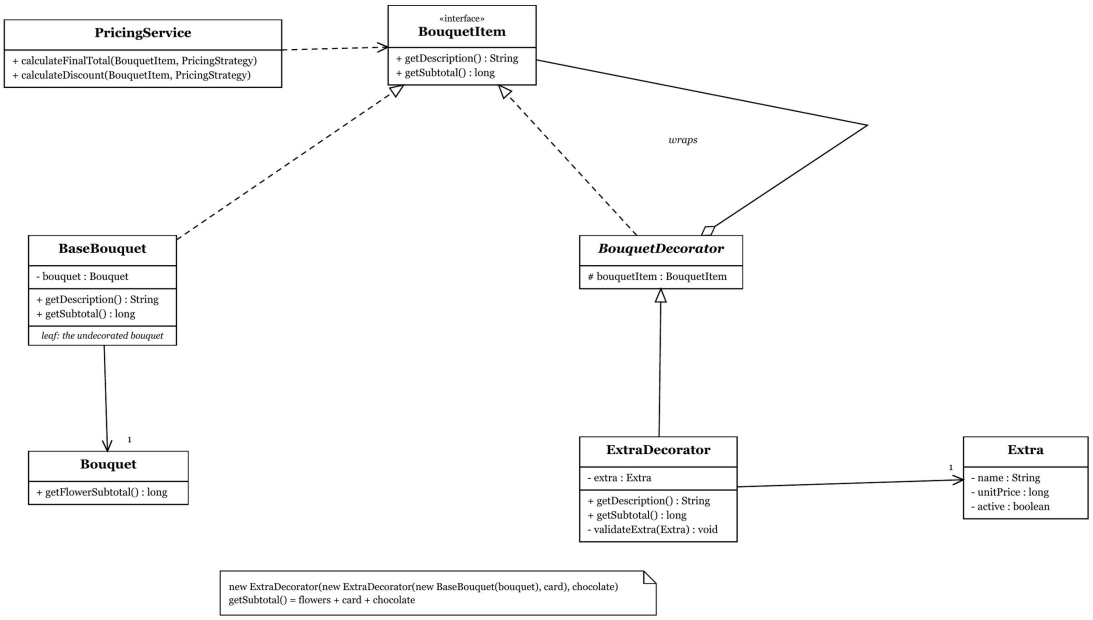
A bouquet is assembled over many UI events; a template is copied into an independent builder



Notation:
solid line = association · open arrowhead = navigable · hollow diamond = aggregation
dashed line + hollow triangle = implements · solid line + hollow triangle = extends
dashed line + open arrowhead = uses (dependency) · italic type name = abstract

2. Decorator — optional bouquet extras

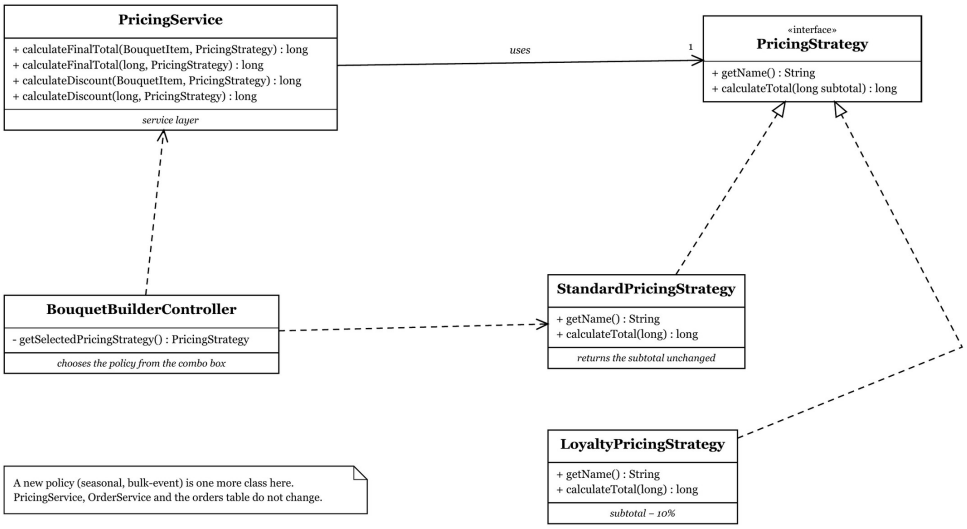
Each extra wraps a BouquetItem and adds to both the description and the subtotal



Notation:
solid line = association · open arrowhead = navigable · hollow diamond = aggregation
dashed line + hollow triangle = implements · solid line + hollow triangle = extends
dashed line + open arrowhead = uses (dependency) · italic type name = abstract

3. Strategy — interchangeable pricing policies

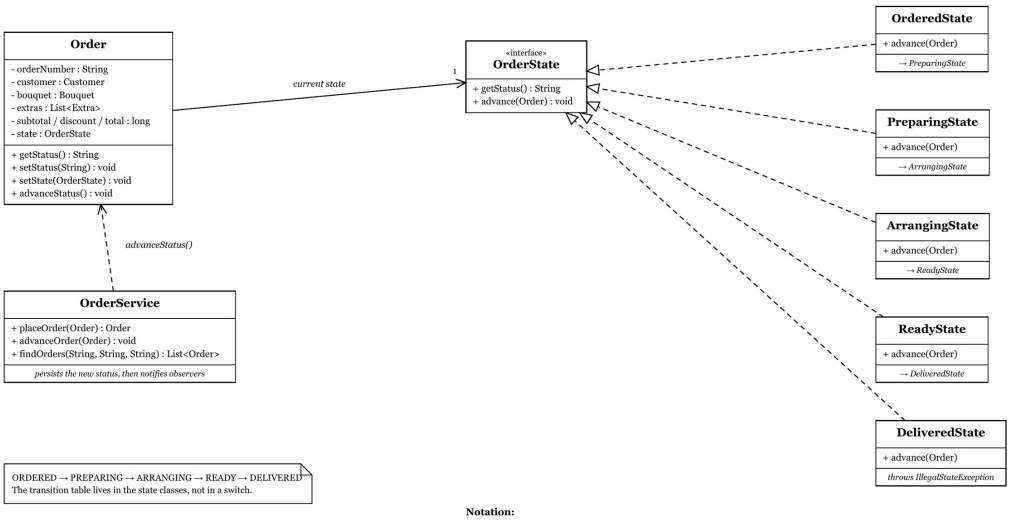
Checkout depends on the PricingStrategy interface, never on a concrete policy



Notation:
solid line = association · open arrowhead = navigable · hollow diamond = aggregation
dashed line + hollow triangle = implements · solid line + hollow triangle = extends
dashed line + open arrowhead = uses (dependency) · italic type name = abstract

4. State — order lifecycle

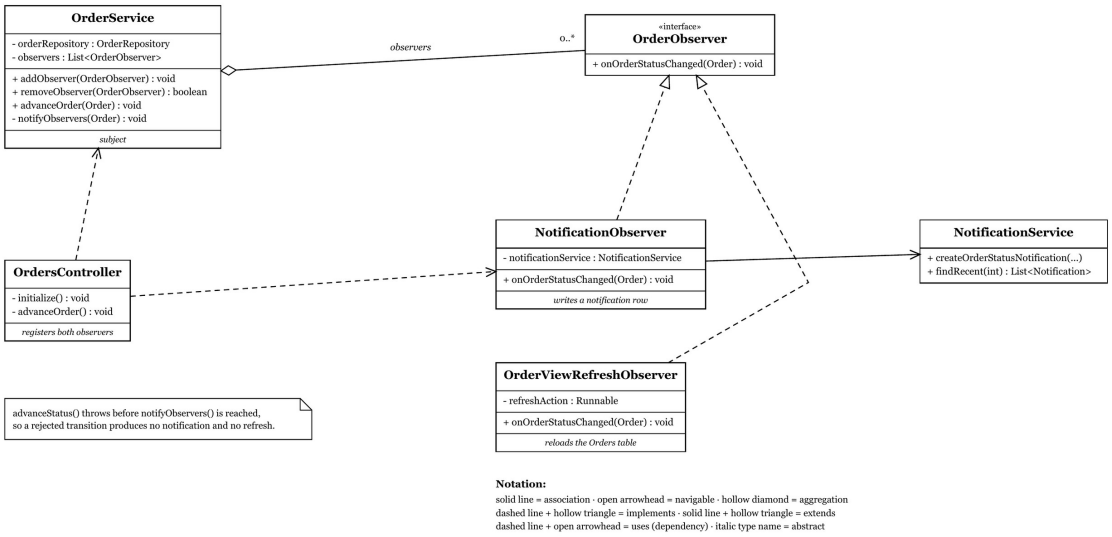
Each state knows only its own successor; DeliveredState refuses to advance



Notation:
solid line = association · open arrowhead = navigable · hollow diamond = aggregation
dashed line + hollow triangle = implements · solid line + hollow triangle = extends
dashed line + open arrowhead = uses (dependency) · italic type name = abstract

5. Observer — reactions to a successful status change

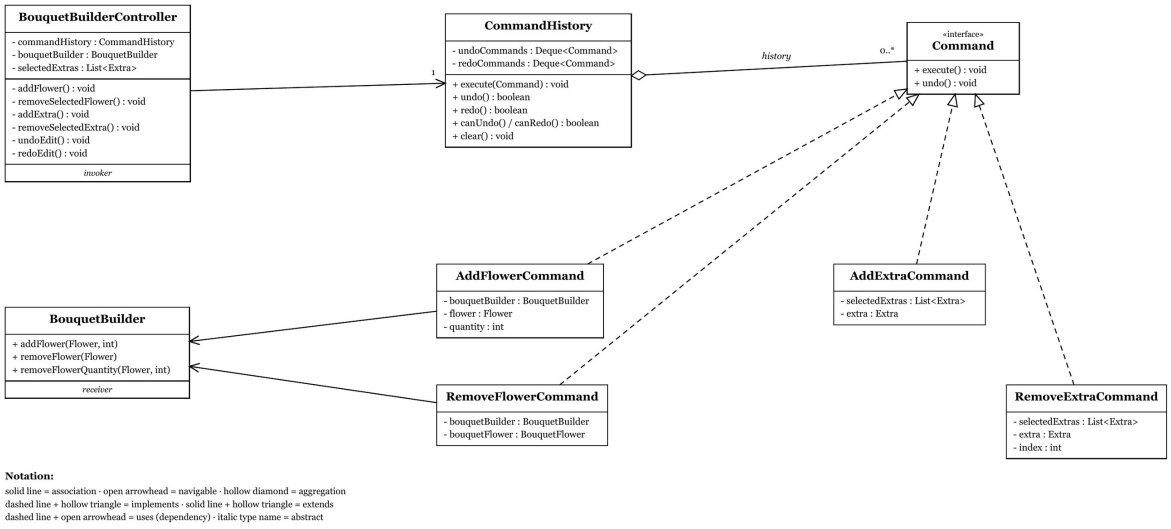
OrderService publishes the change; observers react without knowing about each other



Notation:
solid line = association · open arrowhead = navigable · hollow diamond = aggregation
dashed line + hollow triangle = implements · solid line + hollow triangle = extends
dashed line + open arrowhead = uses (dependency) · italic type name = abstract

6. Command — undo and redo in the bouquet editor

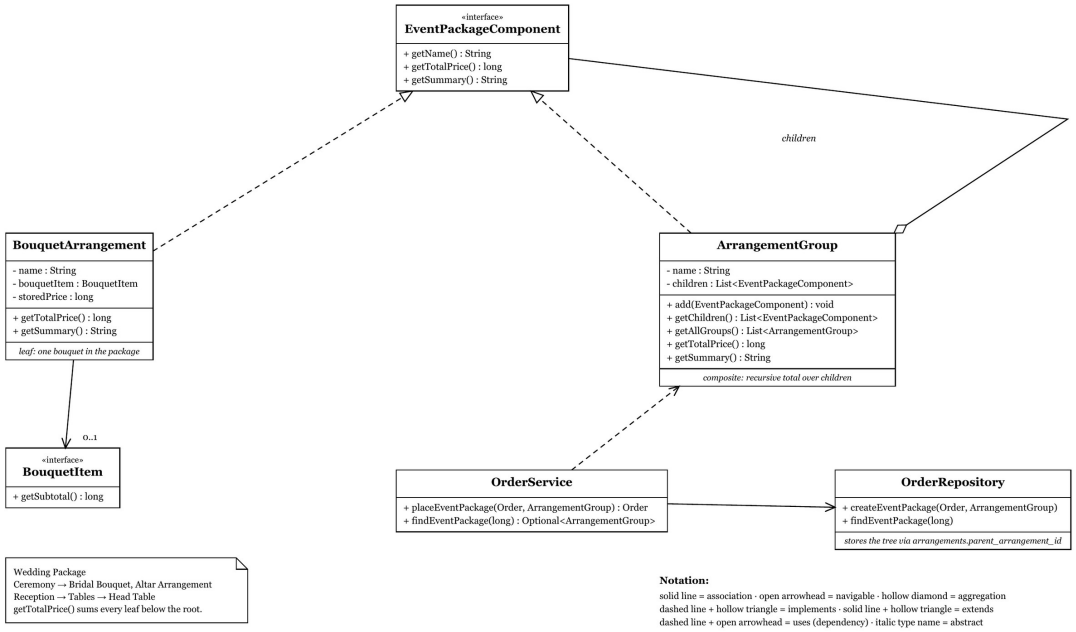
Every reversible editor action is an object with execute() and undo()



Notation:
solid line = association · open arrowhead = navigable · hollow diamond = aggregation
dashed line + hollow triangle = implements · solid line + hollow triangle = extends
dashed line + open arrowhead = uses (dependency) · italic type name = abstract

7. Composite — nested event packages

A group answers getTotalPrice() by asking its children, at any depth



8. Layered structure

controller → service → domain / pattern → repository → SQLite; dependencies point one way only